An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary retains comparisons in directed acyclic graphs and represents possible orders by a region of real positions satisfying interval bounds and the recorded inequalities. It estimates the fraction of this region's volume retained by each answer through deterministic calculations on bounded graph neighborhoods. Moving items into ordered subproblems controls graph degrees and directed path lengths; a geometric analysis shows that these moves preserve the optimal asymptotic bound.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys.[1] A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them.

A binary comparison tree must distinguish all $n!$ total orders, giving the *information-theoretic lower bound* (ITLB) of $\lceil \log(n!) \rceil = n \log n - O(n)$ comparisons in the worst case [1, Section 5.3.1]. This bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Kaplan, Zamir, and Zwick [7, Section 7] ask whether an efficient deterministic adversary can force $(1-\varepsilon)n \log n$ comparisons for every $\varepsilon > 0$ and all sufficiently large n. The following theorem answers their question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that uses polynomially many bit operations per query and forces every correct sorter to make at least $(1 - o(1))n \log n$ comparisons.*

[1] Throughout, log denotes $\log_2$, and ln the natural logarithm. We assume $n \geq 2$.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [10].

Deterministic adversaries from graph entropy. Kahn and Kim [12] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \leq q) \leq \frac{2^q}{n!}.$$

Thus $C \geq \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Section 2 develops the construction from the larger-half rule. The formal adversary (Section 3) retains comparisons in graphs at the nodes of a tree. Section 4 bounds the number of moves and counts orders consistent with the answers. Sections 5 and 6 give the local calculation and the polynomial running-time bound.

Section 7 applies the same prefix bound to arranging keys into a prescribed partial order, with a vanishing relative loss from the information bound. For sorting from supplied initial relations P, a reduction forces $(1 - o(1)) \log e(P) - O(n \log \log n)$ further comparisons. Appendix G proves both consequences. Section 8 reviews earlier adversaries. The manuscript source is available online.[2]

[2] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

2 Developing the adversary

2.1 Keeping the comparison graph small

Start with the larger-half rule. Give each key x a position $X_x \in (0,1)$ and record a comparison answered $x < y$ by imposing $X_x < X_y$. Let P be their partial order. Its feasible region is

$$K(P) = \{X \in (0,1)^n : X_x < X_y \text{ whenever } x <_P y\}.$$

Each total order occupies volume $1/n!$, so $\mathrm{vol}(K(P)) = e(P)/n!$ [9]. To decide which answer retains more orders, compare the volumes on the two sides of $X_x = X_y$. Dyer, Frieze, and Kannan [11] approximate such volumes in randomized polynomial time. We need a deterministic way to choose an answer that keeps almost half the volume.

We record comparisons as arcs and use finite calculations around the queried vertices. A vertex with many neighbors makes even one coordinate update depend on many other positions. A long directed path causes a different problem. In a pass that updates predecessors before successors, a change can travel through the whole path in one pass. Bounding both degree and directed path length limits how far a calculation can reach and how many vertices it needs to visit. As comparisons accumulate, we need a way to remove vertices while preserving the answers they record.

2.2 Fixing a position and continuing in a child

Choose a point c inside the range of feasible positions of a key x. Fixing $X_x = c$ turns each incoming comparison $y < x$ into the bound $X_y < c$ and each outgoing comparison $x < z$ into $c < X_z$. We can remove the vertex of x and keep those bounds.

The key still needs a place in the order. Put it in a block anchored at c. Positions below c precede the whole block, and positions above c follow it. Several keys can enter the block as long as their placement respects earlier answers. Give each a fresh position in $(0,1)$ to describe its order inside the block. That block is a child node with its own comparison graph, so the same operation can be used there. Each position describes an item's order at the node that stores it.

For example, start with $0 < X_A < X_B < 1$, a triangle of area $1/2$. Fixing $X_A = 1/4$ leaves $X_B \in (1/4, 1)$. Give A a fresh child coordinate $U_A \in (0,1)$. The new feasible assignments form a rectangle of area $3/4$ (Figure 1). The volume grows by a factor of $3/2$, and both regions represent just the order $A < B$.

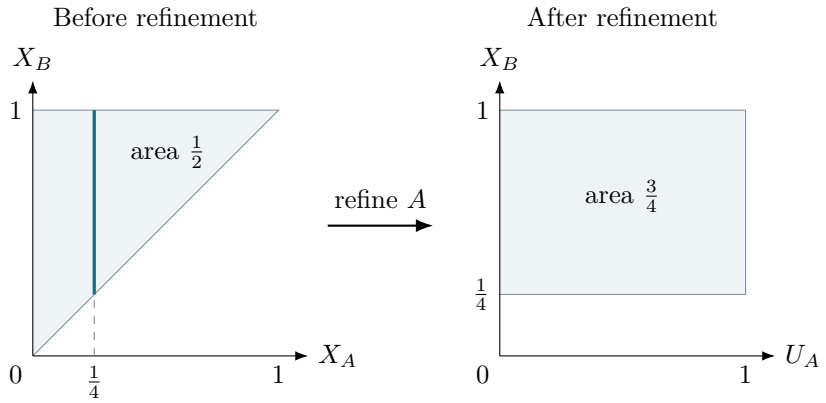


Figure 1: The highlighted section of the triangle has length $3/4$. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is $3/2$ times the original area.

The factor 3/2 is the length of the section at $X_A = 1/4$ divided by the area of the triangle. This ratio is the density of X_A at the chosen point under the uniform distribution on feasible positions. The same identity holds for a general move (Lemma 3.1). It gives us a reason to look for anchors of high density.

Once keys move into blocks, the initial identity between volume and order count no longer applies. The counting proof must account for the locations of the blocks as well as the volume inside each node. A finer binary location takes more bits to specify. We choose anchors so that their volume gains help pay for this cost, and bound how often the adversary needs to move a key.

3 The new optimal adversary

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. Figure 2 shows an example.

Each item has a record (x, ℓ_x, r_x) stored at exactly one node. The label x identifies the key. Its position variable X_x ranges over the open interval $(\ell_x, r_x) \subseteq (0, 1)$. The bounds ℓ_x, r_x are fractions whose denominators are powers of two. We call such numbers *dyadic rationals*. We call an item *free* at its node while its position remains variable.

In the node's DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. Each answered comparison between free items contributes at most one arc.

A *feasible assignment* at a node is a choice of real values for its position variables satisfying $\ell_x < X_x < r_x$ for every item x and $X_x < X_y$ for every arc $x \to y$ in its graph.

We keep each stored interval equal to the range of values its coordinate can take in feasible assignments. An arc $x \to y$ propagates lower bounds from x to y and upper bounds from y to x. After changing a node, the routine `Normalize` makes one pass in topological order for the lower bounds and one in reverse order for the upper bounds. Algorithm 6 gives the two passes. Appendix A proves that they preserve exactly the same feasible assignments.

Initially the tree consists of just the root, containing all records with bounds $\ell_x = 0$ and $r_x = 1$. The position variables X_x are unassigned, and the root's graph has no arcs. Thus every assignment in $(0, 1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0, 1)$ in its parent, called its *anchor*. A node has at most one child at each anchor. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0, 1)$, and its in-node DAG and interval bounds describe the possible orders inside this block.

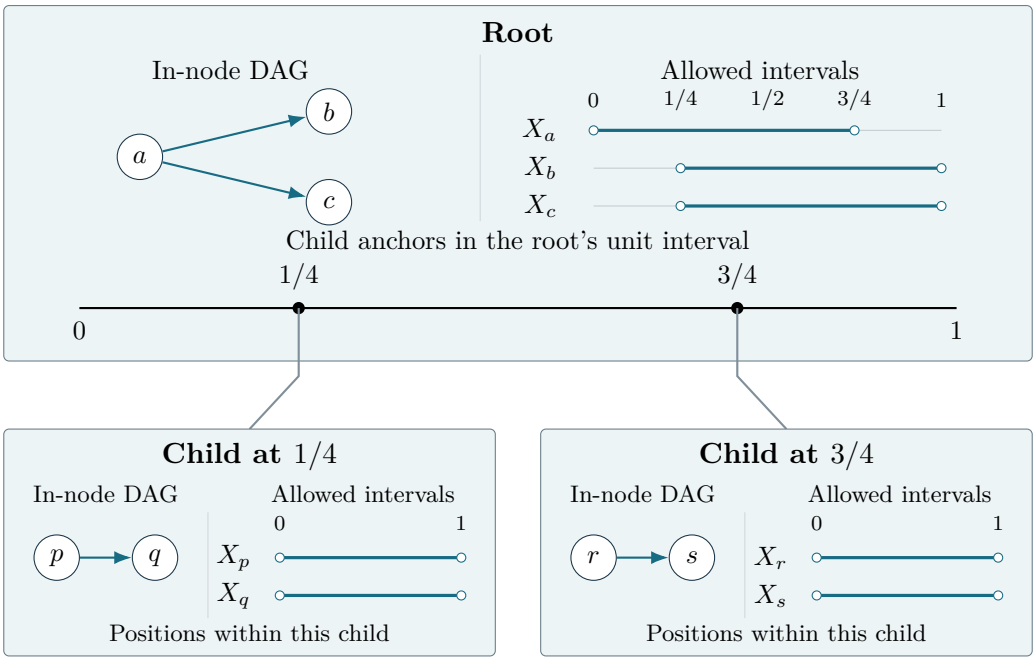


Figure 2: A state of the adversary. Each large box is a tree node, containing an in-node DAG and one open interval for each of its items.

In the root of Figure 2, the in-node DAG records $a < b$ and $a < c$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. The *transcript* is the sequence of answers so far. Our invariant is that every represented order satisfies the transcript.

3.2 Moving an item into a child

The operation `RefineAt`(x, c) fixes the position of x at an interior point $c \in (\ell_x, r_x)$ in its current node.

Fixing $X_x = c$ places x in the child block anchored at c. The arcs incident to x in the parent graph become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, and retain the new bounds on its neighbors. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh position variable with interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 3 illustrates reuse of an existing child.

Algorithm 1: `RefineAt`(x, c): move an item into a child.

Input: A free item x, an interior anchor c, and the current adversary state.
Output: The updated state, with x stored in a child of its former node.
$v \leftarrow$ the node storing x
foreach *arc* $y \to x$ *in the graph at* v **do**
  $r_y \leftarrow \min(r_y, c)$
foreach *arc* $x \to z$ *in the graph at* v **do**
  $\ell_z \leftarrow \max(\ell_z, c)$
Remove x's record from v and its vertex and incident arcs from the graph at v
`Normalize`(v)
if v *has no child anchored at* c **then**
  Create a child of v at c, with no records and an empty graph
$u \leftarrow$ the child of v anchored at c
Insert a record $(x, 0, 1)$ at u
Insert x as an isolated vertex in the graph at u

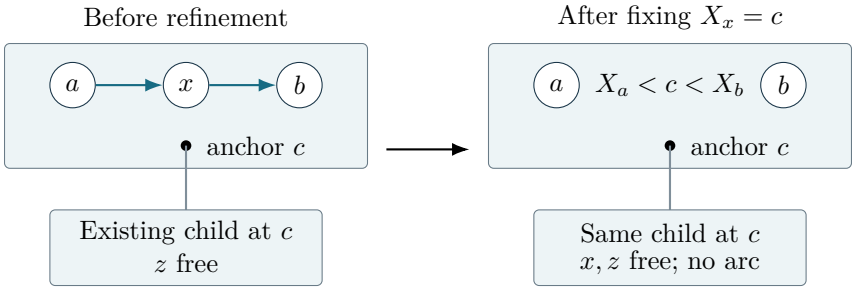


Figure 3: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

At node v, the feasible assignments form the region

$$K_v = \{X : \ell_x < X_x < r_x \text{ for every free } x, \quad X_x < X_y \text{ for every arc } x \to y\}. \tag{1}$$

Write $V_v = \mathrm{vol}(K_v)$ and take the product over the adversary's tree:

$$Z = \prod_v V_v. \tag{2}$$

Each item has a position variable at exactly one node, so the product region has dimension n. A node with no free items contributes volume one. Initially $Z = 1$; whenever every node has positive volume, $0 < Z \le 1$.

Refinement fixes an item's position in its parent and gives it a fresh unit coordinate in a child. Write

$$g = \log \frac{Z_{\text{after}}}{Z_{\text{before}}} \tag{3}$$

for the resulting change in $\log Z$. We call g the refinement's *volume gain*; it can be negative.

To evaluate this gain for an item x stored at node v, let $S_x(c)$ be the volume of the section of K_v obtained by fixing $X_x = c$, measured in the remaining coordinates. Under the uniform distribution on K_v, the coordinate X_x has density

$$f_x(c) = \frac{S_x(c)}{V_v}, \qquad \mathbb{P}(a < X_x < b) = \int_a^b f_x(t)\, dt.$$

Lemma 3.1 (Refinement identity)**.** *Suppose every node region has positive volume. Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{4}$$

Proof. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0,1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0,1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. Normalizing the interval bounds preserves these regions. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

Thus the refinement's volume gain is $g = \log f_x(c)$.

Lemma 3.2 (Refinement and child admission)**.** *Suppose the state represents only orders consistent with the transcript and every node has positive feasible volume. An interior refinement preserves both properties.*

Proof. Take feasible positions representing any order after the move. Remove the fresh coordinate of x from the child and restore $X_x = c$ in the parent. The parent positions satisfy the old inequalities by construction. Keeping the other coordinates fixed, we can perturb X_x slightly to either side of c, since its inequalities are strict. These two old orders place x immediately before and immediately after the child block. Both respect the transcript, so it contains no comparison between x and an item in the block. In the new order, x can occur anywhere inside that block, with its order relative to all outside items preserved. For a new child the block was empty, and the same argument places x at its former parent position. Positivity follows from Lemma A.1 and the refinement identity. □

A refinement can change which orders the state represents. For instance, a move into an existing child allows interleavings inside that child that were previously excluded. The invariant concerns consistency with the transcript.

3.3 Answering a comparison

To compare A and B, descend from the root while one child subtree contains both items. At the node where the descent stops, represent each item by its free coordinate if stored there, or by the anchor of its child subtree. Two distinct anchors determine the answer directly. Otherwise, the following routines either settle the comparison or supply an anchor for a refinement.

Choose an integer $k \geq 64$ and put

$$H = 2^k, \qquad \eta = \frac{1}{100k}. \tag{5}$$

Degree counts all neighbors, and path length counts vertices. Between calls the graph at every node has degree below k and path length below H. During a call we allow degree k and paths of up to $2H$ vertices.

We want an answer to cost about one bit of volume, so the comparison routine must retain nearly half the volume. When its test fails, it must give us a move with enough gain to pay for the refinement. Repairs also need a way to choose an anchor for any item selected by the graph rules. The finer the binary location, the more the counting proof must pay for it, so the

guaranteed gain grows with the anchor's precision. To use these guarantees in polynomial time we need a computable limit on the finite enumeration and a bound on the number of bits in each new anchor.

Lemma 3.3 (Comparison and anchor selection)**.** *Consider a node with positive feasible volume, degree at most k, and directed paths of at most $2H$ vertices.*

1. *For two distinct free coordinates, or a free coordinate and a dyadic threshold,* `Select` *returns either an answer retaining at least $1/2 - \eta$ of the volume or an interior dyadic anchor for a queried free coordinate with refinement gain $g > k$.*
2. *Given any free coordinate,* `ChooseAnchor` *returns an interior dyadic anchor.*

For either routine, write the returned anchor in reduced form $c = (2i+1)/2^{m+1}$. Its gain satisfies

$$g \geq (1 - 1/k)m - 4. \tag{6}$$

The routines are deterministic. Each enumerates at most E_k digit assignments for a computable budget E_k depending only on k. If the stored bounds have at most Λ fractional binary digits, each returned anchor has at most $\Lambda + \log E_k$ such digits.

The finite calculation in Section 5 implements these routines. Appendix D proves the guarantees.

A comparison between two free items adds an arc. A comparison between a free item and an anchor tightens one interval bound. A degree repair removes k arcs inserted by earlier comparisons, including the arc that triggered the repair. In a path repair, moving one item sets a lower bound on the next item's position. The anchors increase along the path. Moving the whole path gives 2^k distinct anchors, so many of them require fine binary locations and bring large gains.

After recording the answer, repair its node if there is a path on H vertices or a vertex of degree k. Give the path priority. Choose the lexicographically first eligible path and refine every item on it in path order, including those whose arcs have already been removed. For a degree repair, refine the smallest eligible label. Each repair anchor comes from `ChooseAnchor`. Use the smallest eligible label for a refinement returned by `Select`.

The adversary keeps the recursive state for at most

$$Q(n) = n \lceil \log n \rceil \tag{7}$$

answers. On the next query it chooses fixed ranks by topologically sorting the transcript, with ties broken by label.

Algorithm 2: `Compare`(A, B): answer a comparison query.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.
if *fixed ranks have been chosen* **then**
  return their order of A, B
if $|\mathcal{T}| = Q(n)$ **then**
  Form the DAG of answers in $\mathcal{T}$ and choose a topological order, breaking ties by label
  Store its ranks and **return** their order of A, B
while *true* **do**
  $v \leftarrow$ the root
  while *A and B lie in the same child subtree of v* **do**
    $v \leftarrow$ that child
  if *A and B lie in different child subtrees of v* **then**
    answer $\leftarrow$ the left-to-right order of their child anchors
    break
  $r \leftarrow$ `Select`(v, A, B)
  if *r is an answer* **then**
    answer $\leftarrow r$
    break
  $(x, c) \leftarrow r$
  `RefineAt`(x, c)
if *at least one queried item is stored at v* **then**
  Store *answer* as an arc in the graph at v or as an interval bound
  `Normalize`(v)
Append *answer* to $\mathcal{T}$
`Repair`(v)
return *answer*

Algorithm 3: `Repair`(v): restore the graph bounds after an answer.

Input: The node v at which the answer was determined.
if *the graph at v has a directed path on H vertices* **then**
  $L \leftarrow$ the selected path's list of labels, in path order
  foreach *$x \in L$ in path order* **do**
    $c \leftarrow$ `ChooseAnchor`(x)
    `RefineAt`(x, c)
else if *a vertex at v has degree k* **then**
  $x \leftarrow$ the selected such vertex
  $c \leftarrow$ `ChooseAnchor`(x)
  `RefineAt`(x, c)

Lemma 3.4 (One-arc repair)**.** *Suppose all node graphs have degree below k and directed paths with fewer than H vertices before an answer is recorded. Recording the answer and executing* `Repair` *restores these bounds. During this operation, degrees are at most k and directed paths have fewer than $2H$ vertices.*

Proof. An answer involving an anchor adds only an interval bound. An answer between free items adds at most one arc, say $x \to y$. Its certified positive probability ensures that the resulting region is nonempty, so the arc cannot create a directed cycle. Only x and y can reach degree k.

Every new path consists of two old paths joined by this arc, so it has fewer than $2H$ vertices.

If an H-vertex path now exists, it must contain $x \to y$. Refining its entire vertex set removes both endpoints. Otherwise a degree repair removes an endpoint that has reached degree k. Either repair deletes the new arc; the remaining parent DAG is a subgraph of the previously valid DAG. Each child receives only isolated vertices, so its bounds remain valid too. If neither violation occurs, no repair is needed. □

Before recording the answer, only queried labels can move. After moving, a label has an isolated coordinate on $(0, 1)$. Further refinements preserve this interval and leave it isolated. Its density is one, so refining it again would have gain zero, contradicting the failure guarantee in Lemma 3.3. The loop thus makes at most two refinements and three calls to `Select`. After the answer, the repair makes at most n further moves.

Proposition 3.5 (Consistency and termination)**.** *Throughout the recursive phase the state has positive feasible volume, and every represented order satisfies the transcript. Each call terminates with at most $n + 2$ refinements. The strict degree and path bounds hold between calls.*

Appendix D gives the induction over the state updates.

4 Counting the orders that remain

Let T_q count the total orders consistent with the first q answers. Every represented order contributes to T_q, which also counts any compatible orders outside the state.

Theorem 4.1 (Comparison prefixes)**.** *For every integer $k \geq 64$, the adversary with parameter k is deterministic and legal. There is a nonnegative $\delta_k = O(\log k/k)$, independent of n and the query sequence, such that after any q answers*

$$\log T_q \geq \log(n!) - (1 + \delta_k)q. \tag{8}$$

To prove the bound we account for two changes. Each comparison can reduce the volume of feasible positions. Each refinement changes the tree locations of the keys, and following those locations has a cost in the order count. Volume gains from refinement help pay for the new locations. We first bound how many times this cost is incurred.

4.1 Refinements are rare

Recording an answer between two anchors leaves Z unchanged. Every other answer retains at least $1/2 - \eta$ of its node's volume, so its loss in $\log Z$ is at most

$$a_\eta = -\log(1/2 - \eta) = 1 + O(1/k). \tag{9}$$

Let G be the sum of the refinement gains through q completed calls. Since Z starts at one and never exceeds one,

$$G \leq \log Z + a_\eta q \leq a_\eta q. \tag{10}$$

Lemma 4.2 (Refinement count)**.** *After q completed recursive calls, the total number P of refinements is at most Cq/k for an absolute constant C.*

Proof. Let D count degree repairs. Each removes k arcs, and each call inserts at most one arc. Thus $D \leq q/k$. By (6), each such move gains at least -4. A move after a failed comparison test gains more than k.

In a path repair, refining x_i at c_i changes the arc to x_{i+1} into the bound $X_{x_{i+1}} > c_i$. The next anchor therefore satisfies $c_{i+1} > c_i$. These $H = 2^k$ anchors are distinct. For reduced denominator 2^{m+1} there are 2^m anchors, so fewer than $H/2$ have $m < k-1$. The batch gains at least

$$(1-1/k)\frac{H}{2}(k-1) - 4H = H\left(\frac{k}{2} - 5 + \frac{1}{2k}\right) \geq \frac{kH}{4}.$$

Every counted batch is complete. Summing the gains gives

$$\frac{k}{4}(P-D) - 4D \leq G \leq a_\eta q, \qquad D \leq q/k.$$

Rearranging proves the claim, for example with $C = 8$ since $k \geq 64$. □

4.2 Counting with the queried labels

For example, if $a < b$ lie in the same child block and z is at the root, the state cannot place z between a and b. If the sole recorded relation among them is $a < b$, the transcript permits $a < z < b$ as well.

Let s be the number of labels that have appeared in queries. Every other label is still an isolated root coordinate on $(0, 1)$. Moves before answering select queried labels, and every item moved in a repair is incident to an arc of the graph before that repair starts. All those arcs were created by comparisons. Normalization leaves an isolated root coordinate unchanged. Temporarily discard the unqueried labels. Their independent unit intervals contribute a factor of one to Z, so the product volume is the same for the remaining s labels.

To count orders of the s queried labels, draw a position for each label independently from the same distribution on the tree. Comparing the positions gives a uniform relative order, since relabelling the draws leaves their distribution unchanged and ties have probability zero. Every draw that reproduces the stored locations and satisfies their constraints gives a represented order. The probability of this event is at most the fraction of represented orders.

Start each draw at the root. At a node, reserve probability $1/k$ for entering a child. In that case, choose a level $m \geq 0$ with probability $\frac{1}{k}(1-1/k)^m$, then choose uniformly among the 2^m anchors of reduced denominator 2^{m+1}. The probability allocated to a particular anchor $c = (2i+1)/2^{m+1}$ is

$$w(c) = \frac{1}{k} \cdot \frac{1}{k}(1-1/k)^m \cdot 2^{-m} = \frac{1}{k^2}\left(\frac{1-1/k}{2}\right)^m. \tag{11}$$

If that child exists, enter it and repeat. Stop at the current node with probability $1 - 1/k$, or when the chosen anchor has no child, and draw a uniform coordinate in $(0, 1)$ there.

The probability of following a label's stored path is the product of the weights along that path. Let $\mathcal{W}$ be the product of these path probabilities over all labels. Root labels contribute one, so discarding the unqueried labels leaves $\mathcal{W}$ unchanged.

Lemma 4.3 (Orders consistent with the transcript)**.** *After a completed recursive call,*

$$T_q \geq n!(1-1/k)^s Z\mathcal{W}.$$

The same inequality holds before the first call.

Proof. The probability that all labels follow their stored paths is $\mathcal{W}$. Stopping at each destination has probability at least $1 - 1/k$. The independent uniform coordinates then satisfy the stored constraints with probability Z. The combined event has probability at least $(1-1/k)^s Z\mathcal{W}$ and so yields at least $s!(1-1/k)^s Z\mathcal{W}$ orders on the queried labels. Its orders respect the transcript by Proposition 3.5. Each such order extends to $n!/s!$ full orders by inserting the other labels, whose positions the transcript does not constrain. Multiplying gives the bound. □

4.3 The prefix bound

Proof of Theorem 4.1. In the recursive phase, legality and termination follow from Proposition 3.5. After the horizon, the fixed ranks respect the recorded answers. For $q \geq Q(n) \geq \log(n!)$, the right side of (8) is nonpositive and $T_q \geq 1$.

For an earlier prefix put $d_k = -\log(1 - 1/k)$. Each refinement multiplies $\mathcal{W}$ by the weight of the child it enters. Its cost $\ell(c) = -\log w(c)$ satisfies

$$\ell(c) = (1 + d_k)m + 2\log k \leq ag + b, \qquad a = \frac{1 + d_k}{1 - 1/k}, \quad b = 4a + 2\log k$$

by (6). Thus $a \geq 1$, $a = 1 + O(1/k)$ and $b = O(\log k)$. Taking logarithms in Lemma 4.3, then using (10), gives

$$\begin{aligned} \log T_q &\geq \log(n!) - sd_k + \log Z - \sum_r \ell(c_r) \\ &\geq \log(n!) - sd_k + (1 - a)\log Z - aa_\eta q - bP. \end{aligned}$$

Here $s \leq 2q$, $(1 - a)\log Z \geq 0$, and $P \leq Cq/k$ by Lemma 4.2. We can therefore take $\delta_k = aa_\eta - 1 + 2d_k + Cb/k$. □

When a correct sorter stops, $T_q = 1$. The theorem forces at least $\log(n!)/(1 + \delta_k)$ comparisons. The same prefix bound applies when an algorithm can stop with several orders still possible.

5 Computing comparison certificates

Both selection routines use lists of intervals enclosing the queried positions. A pair of disjoint intervals certifies an order, and a list of intervals can certify how much probability lies on each side of a candidate anchor interval.

5.1 A coordinate update with finitely many choices

Start with a uniform position $r \in (0, 1)$. The updates

$$r \longmapsto r/2, \qquad r \longmapsto (r + 1)/2$$

send it uniformly to the left and right halves of the interval, respectively. Choosing either update with probability $1/2$ therefore leaves the position uniform on $(0, 1)$. With two updates, the four possible sequences give the four quarter intervals. For any particular sequence, we know the quarter that contains the output even if the starting position is unknown.

Apply the same idea to one coordinate while holding its neighbors fixed. Divide the permitted interval into S equal pieces. Each digit chooses a piece and sends the coordinate to the same relative position within that piece.

Let G be the node graph, with its stored canonical interval endpoints ℓ_z, r_z. For a current assignment Y, the interval permitted at coordinate z has endpoints

$$L_z = \max(\{\ell_z\} \cup \{Y_u : u \to z\}), \qquad U_z = \min(\{r_z\} \cup \{Y_u : z \to u\}).$$

Fix a power of two S. For a digit $j \in \{0, \ldots, S - 1\}$, update

$$Y_z \leftarrow \frac{Y_z + (S - 1 - j)L_z + jU_z}{S}. \tag{12}$$

The coefficients are nonnegative and sum to one. Thus an assignment satisfying the interval bounds and arc inequalities weakly continues to do so after an update, including when $L_z = U_z$.

Choosing the digit uniformly preserves the uniform distribution on the feasible region. Start from a uniform feasible assignment and condition on all coordinates other than z. Then Y_z is uniform on (L_z, U_z). Its normalized position $r = (Y_z - L_z)/(U_z - L_z)$ is sent to

$$r' = \frac{r + j}{S}.$$

For a fixed digit, this is uniform on the corresponding one of the S equal subintervals. Averaging over the digits makes r' uniform on $(0, 1)$ again.

5.2 Enclosing intervals from two calculations

Let T be the set of queried coordinates. Retain every vertex within undirected distance ρ of T, together with all arcs between retained vertices. Retain their stored interval endpoints. Vertices at distance exactly ρ form the boundary. The other retained vertices form the interior.

Follow a feasible assignment X drawn uniformly from the node region K in the proof. Update it on the full graph using independent uniform digits. It stays uniform by the coordinate argument above. To enclose its positions, run two endpoint calculations sharing its digits on the interior vertices. Start the lower calculation at $A_z = \ell_z$ and the upper one at $B_z = r_z$. Hold both boundary arrays fixed. A *sweep* updates each interior vertex once in topological order, and all its neighbors are retained.

An update uses the new predecessor values, the old successor values, and its own old value. These dependencies determine the endpoint values for any topological order. For the enclosure proof, use the restriction of the full order to the retained graph. At an interior update, the maxima, minima, and nonnegative coefficients in (12) preserve $A_z \le X_z \le B_z$. At the boundary, the full assignment stays within the fixed bounds. Thus each digit sequence bounds the queried positions of the hidden uniform point. The adversary only runs the endpoint calculations and enumerates their digits.

Enumerate every assignment of digits to t sweeps. If there are b' interior vertices, this gives

$$N = S^{b't} \tag{13}$$

computations. Each contributes the intervals $[A_x, B_x]$, $x \in T$, to the list, including repeated entries. In the proof experiment, each of the N digit sequences is equally likely, so it selects a uniform entry of this list.

Algorithm 4: `Enclose`: compute a finite list of enclosing intervals.

Input: The retained node graph and its stored interval bounds, its interior vertices, a topological order, queried coordinates T, and integers S, t.
Output: A list of enclosing intervals $([A_x, B_x])_{x \in T}$, one per digit assignment.
$\mathcal{L} \leftarrow$ the empty list
foreach *assignment of digits* $j_{z,s} \in \{0, \dots, S-1\}$ *to the* $b't$ *interior visits* **do**
  $A_z \leftarrow \ell_z$ and $B_z \leftarrow r_z$ for every retained vertex z
  for $s = 1, \dots, t$ **do**
    foreach *interior vertex* z *in the supplied order* **do**
      Apply (12) to A_z, using the current A values and digit $j_{z,s}$
      Apply (12) to B_z, using the current B values and the same digit
  Append $([A_x, B_x])_{x \in T}$ to $\mathcal{L}$
return $\mathcal{L}$

5.3 How far a sweep can reach

After t sweeps the queried endpoints agree exactly with a calculation on the full graph, provided the retained neighborhood is large enough. To see how large, trace the dependencies of a queried endpoint.

Within one sweep, a dependency on a new value follows an incoming arc. If directed paths have at most h vertices, this trace follows at most $h-1$ arcs. Moving from that path to an old value crosses at most one more arc. A value after t sweeps thus depends only on digits, bounds and initial values within distance ht. With $\rho = ht+1$, every update in this dependency calculation is in the retained interior. Its initial values, stored bounds, and digits are the same in the local and full calculations. Consequently the queried endpoints agree exactly for every assignment of the digits, with arbitrary digits outside the retained region.

5.4 How narrow the intervals become

The enclosure argument gives a valid interval for every digit sequence. Finite dependence explains where its endpoints can be computed. To certify a comparison, we also need most intervals to be short. Appendix B bounds the widths after full sweeps by a contraction estimate. Exact agreement transfers that bound to the local calculation.

Theorem 5.1 (Local enclosing intervals)**.** *Let $d, h \geq 1$ be integers and let $\delta, \alpha \in (0,1)$ be rational. There are computable parameters S, t, ρ and a bound E, depending only on d, h, δ, α, with the following property.*

Suppose G is a finite DAG with canonical dyadic interval bounds, degree at most d, directed paths of at most h vertices, and a positive-volume feasible region $K \subseteq (0,1)^{V(G)}$. For any nonempty set T of one or two coordinates, Algorithm 4 returns a list satisfying two guarantees:

1. *A uniformly selected entry and a uniform feasible assignment X can be drawn jointly so that $A_x \leq X_x \leq B_x$ for every $x \in T$.*
2. *At most a fraction α of its entries have $\max_{x \in T}(B_x - A_x) > \delta$.*

The calculation enumerates at most E digit assignments. If the stored bounds have at most Λ fractional binary digits, every computed endpoint has at most $\Lambda + \log E$ fractional digits.

5.5 From intervals to answers and anchors

For two queried positions, count the entries with $B_x < A_y$ and those with $B_y < A_x$. For a threshold use $[c, c]$ as its interval. Each count divided by N is a lower bound on the corresponding probability because the enclosed positions obey the certified order. `Select` uses the answer with the larger bound if that bound is at least $1/2 - \eta$. Otherwise it chooses a queried free coordinate and rounds in the middle half of its range.

For a repair, `ChooseAnchor` uses a one-coordinate list. It tests whether a proposed interval contains at least a quarter of the mass, with at least a quarter on either side. The test counts entries lying wholly in each of these three regions. On success it rounds within the tested interval. On failure it rounds in the middle half of the coordinate's range. Appendix C gives the endpoint selection and shows that a failed test identifies a short coordinate interval. The gain and precision estimates in Appendix D complete the proof of Lemma 3.3.

Algorithm 5: `ChooseAnchor`(x): choose a position for refinement.

Input: A free item x at node v, with stored interval (ℓ_x, r_x).
Output: A dyadic anchor $c \in (\ell_x, r_x)$.
$J \leftarrow$ `CentralInterval`(v, x)
if J *is* `FAIL` **then**
  $W_x \leftarrow r_x - \ell_x$
  $J \leftarrow [\ell_x + W_x/4,\ r_x - W_x/4]$
return `RoundUp`(J)

6 A uniform polynomial-time implementation

For each fixed degree threshold k, the preceding sections give a finite deterministic adversary. We now bound how long it maintains the recursive state and choose k slowly enough that its local calculations take polynomial time. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

6.1 A finite horizon for the recursive state

Recall the horizon $Q(n) = n\lceil \log n \rceil$ from (7). The algorithm runs the recursive adversary for at most $Q(n)$ answers, recording this prefix of the transcript. If another query arrives, it forms the directed graph of those answers and chooses its lexicographically first topological ordering. It answers that query and every subsequent query from the resulting fixed ranks.

Proposition 3.5 ensures that the prefix is consistent with a total order, so this graph is acyclic and the topological ordering exists. Every later answer remains consistent with the prefix. Lemma 4.2 bounds the number of refinements after q completed calls by $O(q/k)$. An unfinished call adds at most $n + 2$: two while selecting an answer and at most n in the structural repair. Thus the number P of refinements before the horizon, including any call still in progress, satisfies

$$P = O(Q(n)/k + n). \tag{14}$$

The recursive tree has at most $P + 1$ nodes and contains n item records. The stored transcript has at most $Q(n)$ pairs. After switching to fixed ranks, the recursive state is no longer needed.

6.2 An explicit budget for the finite calculations

Appendix C chooses the parameters in the local enclosure theorem. Write S_k for its number of digit choices, t_k for the number of sweeps, and b_k for the bound on retained vertices in (29). The number of digit sequences in one calculation is at most

$$E_k = S_k^{b_k t_k}. \tag{15}$$

For $k = 64, 128, \ldots$, a fixed program writes the complete parameter tuple and E_k in binary. On input size n, simulate this program for n steps. Among the tuples it has completed, choose the largest k with

$$E_k \le n. \tag{16}$$

Use these parameters for the entire recursive phase. If no tuple qualifies, answer all comparisons according to the fixed label order.

The initialization takes polynomial time: the simulation writes at most n bits, and its completed tuples can be parsed and compared within a polynomial bound. For every fixed k, the program finishes its tuple in some finite time $T(k)$. Once $n \ge \max\{T(k), E_k\}$, that tuple is available and meets the budget. Hence the selected $k(n)$ tends to infinity.

6.3 Precision and work per call

Each finite calculation enumerates at most $E_k \le n$ digit assignments. For each assignment, either endpoint calculation uses at most $\log E_k$ updates, so the work involves polynomially many arithmetic operations. A comparison call uses at most three comparison calculations and at most n central-interval calculations, one for each structural refinement.

The numbers also have polynomial length. Let Λ be the greatest number of fractional binary digits in any stored endpoint or anchor. A refinement increases this by at most $\log E_k$, and normalization only copies existing bounds. Hence

$$\Lambda \le P \log E_k = O(n \log^2 n). \tag{17}$$

Appendix E proves this precision bound and accounts for the graph operations, rounding, and exact arithmetic. Together with the finite horizon, these bounds give one fixed polynomial for the work per query, independent of the selected parameters and the length of the interaction. After the horizon, each answer compares two stored ranks.

Proof of Theorem 1.1. Initialization and every comparison call are deterministic and take polynomial time. The recursive phase is legal by Proposition 3.5, and the final fixed order respects its recorded answers. For all sufficiently large n, a parameter tuple qualifies. When a correct sorter stops, $T_q = 1$. Theorem 4.1 then gives

$$q \ge \frac{\log(n!)}{1 + \delta_{k(n)}} = (1 - o(1)) n \log n.$$

□

7 Consequences and further questions

The counting argument also applies when the algorithm starts or finishes with a partial order. The two settings have different information bounds: initial relations reduce what remains to be learned, whereas a target partial order specifies what the algorithm must establish. We state the consequences here and give their proofs in Appendix G.

7.1 Producing a prescribed partial order

Let R be a partial order on n slots. In *partial-order production*, the input consists of n keys with no known relations. The algorithm must assign one key to each slot so that the key in slot u is smaller than the key in slot v whenever $u <_R v$. Sorting is the case where R is a chain; a slot required to precede every other slot asks for the minimum.

A fixed assignment is valid for exactly $e(R)$ total orders of the keys. The information-theoretic lower bound is therefore

$$I(R) = \log(n!) - \log e(R).$$

Cardinal et al. [14] give a deterministic polynomial-time algorithm using $I(R) + o(I(R)) + O(n)$ comparisons.

Corollary 7.1 (Partial-order production)**.** *There is a nonnegative sequence $\varepsilon_n \to 0$, independent of the target, such that the adversary of Theorem 1.1 forces every correct comparison algorithm producing an n-element partial order R to make at least*

$$(1 - \varepsilon_n) I(R)$$

comparisons. The adversary is initialized using only n and works for every target R.

Its leading term matches Cardinal et al.'s upper bound when $I(R) = \omega(n)$. The information bound can be smaller than the task's comparison complexity. For example, producing a minimum has $I(R) = \log n$ and requires $n - 1$ comparisons.

7.2 Sorting from supplied partial information

Now let P describe relations already known between the input keys, supplied as a DAG. The sorter must identify one of its $e(P)$ linear extensions, so the information bound is $\log e(P)$.

Corollary 7.2 (Supplied partial information)**.** *There is a deterministic adversary that can be initialized from any n-element partial order P in polynomial time, gives legal answers in polynomial time per comparison, and forces every correct sorter to make at least*

$$(1 - o(1)) \log e(P) - O(n \log \log n)$$

additional comparisons. Both error terms are uniform over P.

The construction uses Cardinal et al.'s polynomial-time procedure [14] to extend P to a *weak order*: a sequence of antichain layers, with every key in an earlier layer preceding every key in a later one. The additional relations reduce $\log e(P)$ by at most $O(n \log \log n)$. Run an independent copy of our adversary within each layer and answer between layers from their fixed order. All copies use the same parameters, chosen from the total size n, so the guarantee also covers small layers.

The comparison bound approaches $\log e(P)$ when $\log e(P) = \omega(n \log \log n)$. Kahn and Kim's adversary already forces $\Omega(\log e(P))$ for every P. The improvement here is the coefficient approaching one in this regime.

7.3 Preserving small amounts of remaining uncertainty

Can a deterministic polynomial-time adversary force $(1 - o(1)) \log e(P)$ further comparisons for every supplied partial order P? The additive loss in Corollary 7.2 can overwhelm the information that remains. Two disjoint chains of length $n/2$ have $\log e(P) = \Theta(n)$; a chain on $n - 1$ items and one unrelated item have $e(P) = n$.

Reducing the additive loss to $O(n)$ would already extend the useful range. Cardinal et al. [14, Section 4] exhibit partial orders P for which every weak-order extension W satisfies $\log e(W) \le \log e(P) - \Omega(n \log \log n)$. A sharper construction must therefore preserve more of the possible interleavings between layers.

Initial partial information also arises when sorting points whose locations are known only to lie in specified intervals [18, 19]. Extending our adversary to those models would require answers compatible with realizable coordinates and with the permitted geometric operations. Another direction is the amortized minimum-maintenance problem linked to sorting adversaries by Kaplan, Zamir, and Zwick [7, Section 7]: handling insertions and deletions would require controlling how they change the represented orders.

8 Related Work

8.1 Sorting with partial information

Fredman [2] studied how closely sorting can approach the information-theoretic lower bound when some orders have already been excluded. For any nonempty family Γ of permutations of n items, he showed that the unknown order can be identified using at most $\log |\Gamma| + 2n$ comparisons. In particular, if the available information is a partial order P, this gives $\log e(P) + 2n$ comparisons,

where $e(P)$ counts its linear extensions. The comparison bound is within an additive $2n$ of the ITLB, but constructing the strategy requires enumerative information about the possible orders and does not give a polynomial-time algorithm for general partial orders [12, p. 390].

Kahn and Kim [12] later gave a deterministic polynomial-time sorting algorithm using $O(\log e(P))$ comparisons. They also constructed a deterministic polynomial-time adversary forcing $\Omega(\log e(P))$ comparisons. This history highlights the distinction between comparison complexity and the computation needed to choose comparisons. Our adversary faces the corresponding task of choosing answers efficiently while forcing asymptotically the full ITLB for ordinary sorting.

For an initial partial order P supplied as a DAG with m arcs, Haeupler et al. [15] give a deterministic algorithm using $O(m+\log e(P))$ time and $O(\log e(P))$ new comparisons. Rutschmann [17] separates this work into $O(m)$ preprocessing and $O(\log e(P))$ sorting time. These results make efficient use of the known relations; Corollary 7.2 supplies an efficient adversary whose comparison coefficient approaches one when $\log e(P) = \omega(n \log\log n)$.

Van der Hoog and Rutschmann [16] study a model with separate oracles for the initial partial order and the unknown total order. They establish tight tradeoffs between preprocessing queries and the later sorting cost. Our reduction processes the full supplied DAG; its guarantee concerns subsequent key comparisons. Lower bounds with restricted access to the initial relations require the additional analysis of their oracle model.

8.2 Binary-tree adversaries

Atallah and Kosaraju [3] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [4] on searching after partial sorting, Richards and Vaidya [5] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [6] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 4 shows the placements.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

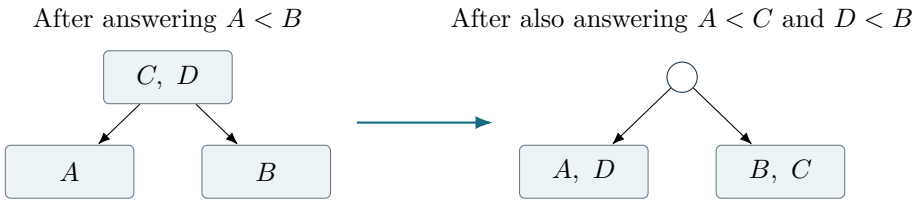


Figure 4: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \geq n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [6, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

8.3 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [8] use a four-child tree. Intermediate states correspond to consecutive ranges of children, so an item can narrow its range before entering a particular child. Each step drops one endpoint of the range (Figure 5). At a tree node, a comparison between two unpaired items can be retained as an ordered pair without moving either item. Retained pairs are disjoint, and their order constrains which children the two items may enter.

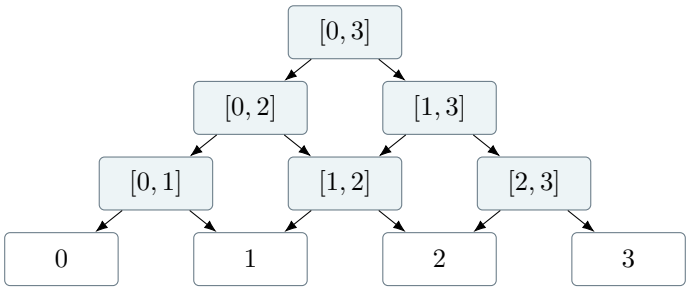


Figure 5: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [8, Section 5]. An intermediate node records a range of possible children. An item at $[1, 3]$ can still enter any of children 1, 2, or 3.

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1)) n \log n$ comparisons [8, Section 6].

A Canonical intervals and densities

We first verify that normalization stores the full range of each coordinate without changing the feasible region. We then establish the density bounds used to choose anchors and analyse probability certificates.

Lemma A.1 (Canonical intervals and sections)**.** *Algorithm 6 preserves the feasible region and its volume. After normalization, the region has positive volume if and only if all stored widths*

$r_x - \ell_x$ are positive. In that case (ℓ_x, r_x) is the projection of the region onto coordinate x, and fixing any $X_x = c \in (\ell_x, r_x)$ leaves a positive-volume section.

Proof. Every propagated bound is already implied by an arc and an interval bound. A forward topological pass propagates every lower bound to all its successors; the reverse pass propagates every upper bound to all its predecessors. Thus normalization preserves the region and gives $\ell_x \le \ell_y$ and $r_x \le r_y$ along every arc $x \to y$.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Along an arc $x \to y$, increasing θ_x to θ_y strictly increases this value, because $r_x > \ell_x$. Replacing the endpoints by ℓ_y, r_y cannot decrease it. Thus $X_x < X_y$, all interval bounds hold strictly, and a small open neighborhood is feasible.

Now fix $X_x = c$ in the stored interval and delete its coordinate. Incident arcs impose the upper bound c at its in-neighbors and the lower bound c at its out-neighbors. Lower bounds propagate forward and upper bounds backward. Any earlier lower bound that reached a successor through x is superseded by $c > \ell_x$; any earlier upper bound that reached a predecessor through x is superseded by $c < r_x$. The normalized intervals of the remaining coordinates are therefore

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases} \tag{18}$$

A predecessor retains positive width because $r_y > \ell_y$ and $c > \ell_x \ge \ell_y$. A successor does too because $r_y > \ell_y$ and $c < r_x \le r_y$. Other widths are unchanged. The criterion just proved shows that the section has positive volume. Since this holds for every $c \in (\ell_x, r_x)$, that interval is exactly the coordinate projection. □

A.1 Bounds from convex sections

Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \ne x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \mathrm{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\mathrm{vol}_N K}.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.2 (Density of a linear projection)**.** *Let $C \subseteq \mathbb{R}^d$ be a bounded open convex region of positive volume, and let φ be a nonzero linear functional with $\varphi(C) = (\ell, r)$ of width W. For X uniform on C, the density g of $\varphi(X)$ satisfies*

$$g(t) \le \frac{d}{W}, \qquad g(t) \ge \frac{4^{-(d-1)}}{W} \quad (\ell + W/4 \le t \le r - W/4).$$

Proof. An invertible linear change of coordinates makes φ the first coordinate. Its constant Jacobian preserves uniformity on the transformed region, so it suffices to prove the bounds for a coordinate projection. Write $S(t)$ for the section volume at that coordinate's value t; then $g(t) = S(t)/\operatorname{vol} C$.

Choose points in $\overline{C}$ whose first coordinates are ℓ and r. Such points exist because C is bounded. For $\ell < t < s < r$, join every point of the section at t to the point at r. Since C is open and convex, the image with first coordinate s lies in the section of C there. Each of its $d-1$ directions is scaled by $(r-s)/(r-t)$, so

$$S(s) \geq \left(\frac{r-s}{r-t}\right)^{d-1} S(t) \qquad (t < s). \tag{19}$$

Joining instead to the point at ℓ gives

$$S(s) \geq \left(\frac{s-\ell}{t-\ell}\right)^{d-1} S(t) \qquad (s < t). \tag{20}$$

Normalize these inequalities and integrate over s on both sides of t:

$$1 \geq g(t)\left(\int_\ell^t \left(\frac{s-\ell}{t-\ell}\right)^{d-1} ds + \int_t^r \left(\frac{r-s}{r-t}\right)^{d-1} ds\right) = \frac{W g(t)}{d}.$$

Thus $g(t) \leq d/W$.

For the lower bound, choose t_0 with $g(t_0) \geq 1/W$, which exists because g integrates to one over an interval of length W. For any $s \in [\ell + W/4, r - W/4]$, the appropriate ratio in (19) or (20) is at least $1/4$. Consequently

$$g(s) \geq 4^{-(d-1)} g(t_0) \geq \frac{4^{-(d-1)}}{W}.$$

The case $s = t_0$ satisfies the same bound directly. □

Lemma A.3 (Coordinate density bounds)**.** *Suppose a node's feasible region has positive volume, and x has at most B other comparable coordinates and canonical interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \leq \frac{B+1}{W}, \tag{21}$$

$$f_x(t) \geq \frac{4^{-B}}{W} \qquad (\ell + W/4 \leq t \leq r - W/4). \tag{22}$$

Proof. Fix all coordinates incomparable with x, at values leaving a nonempty region. By the section formula in Lemma A.1, each such fixing leaves x's canonical interval unchanged. The remaining region C is open and convex, has dimension $d \leq B+1$, and still projects onto (ℓ, r) in coordinate x. Lemma A.2 therefore gives both required bounds for the density of x in C.

Average over the fixed coordinates. If ω denotes their values, C_ω the remaining region, and g_ω its coordinate density, integration of each section gives the pointwise identity

$$f_x(t) = \frac{\int \operatorname{vol}(C_\omega) g_\omega(t)\, d\omega}{\int \operatorname{vol}(C_\omega)\, d\omega}.$$

Empty slices contribute zero. The bounds on g_ω are uniform in ω, so they also hold for f_x. □

Lemma A.4 (A bound for small differences)**.** *Let X be uniform on a positive-volume node region, and suppose x has at most B other comparable coordinates and canonical interval of width W. Let Y be a fixed number or X_y for any distinct free coordinate y. For every interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}. \tag{23}$$

Proof. For a fixed number Y, integrate the coordinate-density upper bound over $I + Y$. Now let $Y = X_y$ for any free coordinate $y \neq x$. As above, fix all coordinates incomparable with x. The remaining convex region C has dimension $d \leq B + 1$, and x retains its width W. We show that the projection $X_x - X_y$ has width at least W in each such region.

If y is incomparable with x, it is now fixed, so the projection is a translate of X_x and has width W. Otherwise y remains free. Write (ℓ_z, r_z) for canonical intervals in C; x's endpoints are unchanged. If a directed path goes from x to y, then $r_y \geq r_x$. Fixing $X_y = t \in (\ell_y, r_y)$ leaves x the interval $(\ell_x, \min(r_x, t))$, by Lemma A.1. As t approaches r_y, its width tends to W. On that section the difference $X_x - t$ varies over an interval of the same width. Thus the projection of the whole region C has width at least W. If a directed path goes from y to x, let t approach ℓ_y instead. Here $\ell_y \leq \ell_x$, and the remaining interval $(\max(\ell_x, t), r_x)$ again has width tending to W.

Lemma A.2 bounds the conditional density of the difference by $d/W \leq (B+1)/W$. Integrating over I and averaging over the fixed coordinates proves (23). If both coordinates have at most B comparable coordinates, exchanging x and y gives the same bound with $\max(W_x, W_y)$ in the denominator. □

A.2 Density in a central interval

A density is *log-concave* if its positive support is convex and its logarithm is concave there. The coordinate densities above are log-concave pointwise: the uniform density on K is log-concave, and integrating out the other coordinates preserves this property by Prékopa's marginal theorem [13, Theorem 8]. We finish with the density bound used for positions having substantial probability on each side.

Lemma A.5 (Density at central positions)**.** *Let X have a log-concave density f. If $\mathbb{P}(X \leq c) \geq p$ and $\mathbb{P}(X \geq c) \geq p$ for some $0 < p \leq 1/2$, then*

$$f(c) \geq p \sup_t f(t).$$

Proof. For $s \leq c \leq t$, log-concavity gives

$$f(s)f(t) \leq f(c)f(s + t - c).$$

Indeed, c and $s + t - c$ are complementary convex combinations of s, t; multiply the two log-concavity inequalities. Fixing $t \geq c$ and integrating over $s \leq c$ gives

$$pf(t) \leq f(c) \int_{-\infty}^{c} f(s + t - c)\, ds \leq f(c).$$

Here the first inequality uses the mass at least p to the left of c, and the last uses that a density integrates to one. Fixing $s \leq c$ and integrating over $t \geq c$ instead uses the right-hand mass and gives $pf(s) \leq f(c)$. Taking the supremum over both sides of c proves the claim. □

Proof of Lemma D.2. Put $M = \sup_t f_x(t)$. The inside-mass condition gives $M|J| \geq 1/4$. Every $c \in J$ has probability at least $1/4$ on each side, so Lemma A.5 gives $f_x(c) \geq M/4 \geq 1/(16|J|)$. □

B Why the enclosing intervals become narrow

We analyse lower and upper calculations on the full graph, with a fresh independent uniform digit at every vertex in every sweep. By Section 5.3, the queried endpoints have the same distribution as those in the finite local list.

B.1 The decrease after one sweep

Write $\Delta_z = B_z - A_z$. Superscripts old and new refer to the start of a sweep and the value after a coordinate's update. For a vertex z, the shared digit j_z gives

$$\Delta_z^{\text{new}} \le \frac{1}{S}\Delta_z^{\text{old}} + \frac{S-1-j_z}{S}\max_{u\to z}\Delta_u^{\text{new}} + \frac{j_z}{S}\max_{z\to u}\Delta_u^{\text{old}}, \tag{24}$$

with an empty maximum equal to zero. The stored interval bounds in the two calculations are identical.

Let $B_h(z)$ be the vertices at undirected distance at most h from z. Put $D = \max_{u\in B_h(z)}\Delta_u^{\text{old}}$ and $a_u = (S-1-j_u)/S$. At z, (24) gives

$$\Delta_z^{\text{new}} \le (1-a_z)D + a_z\max_{u\to z}\Delta_u^{\text{new}}.$$

Follow a predecessor attaining the maximum, and continue backwards to a source. This path has at most h vertices; all its vertices and their successors lie in $B_h(z)$. At the source the predecessor term is zero. Substitution along the path bounds the new width by $(1-\prod_u a_u)D$.

Write $\Delta_z(s)$ for the width after s sweeps. Call the pair (z,s) *bad* if, during sweep s, the digit at z or any vertex preceding z on a directed path equals $S-1$. Otherwise all factors a_u in the preceding expansion are at least $1/S$, and there are at most h of them. Put $\beta = 1-S^{-h}$. Consequently,

$$\Delta_z(s) \le \kappa_z(s)\max_{u\in B_h(z)}\Delta_u(s-1), \qquad \kappa_z(s) = \begin{cases} 1, & (z,s) \text{ is bad,} \\ \beta, & \text{otherwise.} \end{cases} \tag{25}$$

B.2 Choosing the parameters

Write $r = 2$ for the bound on the number of queried coordinates. Both $B_h(z)$ and the set consisting of z and its directed-path predecessors have at most M vertices. Choose

$$M = (d+1)^{h+1}, \qquad S = \text{the least power of two at least } 4M^2. \tag{26}$$

The bad event in (25) involves only fresh digits from the current sweep. Its conditional probability is at most $M/S \le 1/(4M)$. With $\beta = 1-S^{-h}$, choose the smallest positive integer p satisfying the rational test

$$\beta^p \le \frac{1}{4M}. \tag{27}$$

Consequently,

$$\mathbb{E}\big[\kappa_z(s)^p \mid \text{earlier sweeps}\big] \le \frac{1}{4M} + \beta^p \le \frac{1}{2M}.$$

This estimate uses all directed-path predecessors in defining the bad event, so it also covers the predecessor choices made in the one-sweep expansion. No independence between vertices within a sweep is required.

Choose the smallest positive integer t satisfying

$$2^{-t} \le \frac{\alpha\delta^p}{r}, \qquad \rho = ht+1. \tag{28}$$

These choices are computable: $0 < \beta < 1$, and both tests compare rational numbers. Retain radius ρ around the queried set T, freezing the outer boundary as in Algorithm 4.

The widths before sweep s are fixed when we condition on earlier sweeps. Raising (25) to the power p, bounding the maximum by a sum, and taking expectations gives

$$\mathbb{E}\Delta_z(s)^p \le \frac{1}{2M}\sum_{u\in B_h(z)}\mathbb{E}\Delta_u(s-1)^p.$$

All initial widths are at most one and $|B_h(z)| \le M$. Induction gives $\mathbb{E}\Delta_z(s)^p \le 2^{-s}$ for every vertex. At sweep t, Markov's inequality gives $\mathbb{P}(\Delta_z(t) > \delta) \le 2^{-t}\delta^{-p} \le \alpha/r$. A union bound over the at most r queried positions proves the width guarantee.

Each radius-ρ neighborhood has at most $1 + d + \cdots + d^\rho \le (d+1)^{\rho+1}$ vertices. We may therefore use

$$b = r(d+1)^{\rho+1}, \qquad E = S^{bt} \tag{29}$$

as bounds on the retained vertices and enumerated digit assignments. All these choices depend only on d, h, δ, α.

C Calibration of the finite certificates

We verify the central-interval routine and choose the finite calculation's parameters. The purpose of the calibration is to ensure that a failed certificate identifies a coordinate whose short interval makes a middle-half refinement profitable.

C.1 The central-interval test

For the one-coordinate list $[A_i, B_i]$, $1 \le i \le N$, take u to be the $N/4$-th upper endpoint in sorted order and v to be the $3N/4$-th lower endpoint. The alphabet size S is divisible by four, and the queried coordinate is interior, so $N/4$ is an integer. Return `FAIL` if $u \ge v$ or fewer than $N/4$ list intervals lie wholly in $[u, v]$. Otherwise return $[u, v]$.

At least $N/4$ intervals end at or below u, and at least $N/4$ begin at or above v. On success, at least $N/4$ intervals also lie wholly inside $[u, v]$. Enclosure certifies mass at least $1/4$ in each of the three regions. The true distribution is continuous and supported on (ℓ_x, r_x), so these masses imply $\ell_x < u < v < r_x$. The endpoints are dyadic, and sorting and counting finite lists are deterministic.

C.2 The width at which certificates succeed

The graph bounds limit how many coordinates can be comparable with a queried coordinate. Define

$$B = (k+1)^{2H+1}. \tag{30}$$

In an node graph of degree at most k and directed paths of at most $2H$ vertices, every comparable coordinate lies within undirected distance $2H$. Thus B bounds the number of other comparable coordinates. Set

$$w_* = 2^{-2kB}. \tag{31}$$

This width calibrates the finite calculations. The algorithms test the computed certificates directly.

Lemma C.1 (When certification succeeds)**.** *Suppose a node has positive feasible volume, its node graph has degree at most k, and its directed paths have at most $2H$ vertices. If any free coordinate involved in a comparison has interval width at least w_*, comparison certification succeeds. If $W_x \ge w_*$, then* `CentralInterval`(v, x) *succeeds.*

Proof of Lemma C.1. Calibrate both routines using Theorem 5.1 with

$$d = k, \qquad h = 2H, \qquad \alpha = \frac{\eta}{2}, \qquad \delta = \frac{\eta w_*}{8(B+1)}. \tag{32}$$

These parameters are fixed for the interaction. They do not depend on the queried widths.

Suppose one queried free coordinate x has $W_x \geq w_*$. If the computed intervals overlap and both have width at most δ, the enclosed positions differ by at most 2δ. Lemma A.4 therefore bounds the unresolved fraction by

$$1 - p - q \leq \alpha + \mathbb{P}(|X_x - X_y| \leq 2\delta) \leq \alpha + \frac{4(B+1)\delta}{w_*} = \eta.$$

Thus $\max(p, q) \geq (1 - \eta)/2 > 1/2 - \eta$, so comparison certification succeeds. For a threshold c, an unresolved interval contains c. If its width is at most δ, its enclosed position is within δ of c, giving the stronger bound

$$\alpha + \frac{2(B+1)\delta}{w_*} = \frac{3\eta}{4}. \tag{33}$$

It remains to show that `CentralInterval` succeeds when $W_x \geq w_*$. By (33), at most a fraction $\tau = 3\eta/4$ of its intervals contain any fixed real number. This bound applies in particular to the selected endpoints u and v. An interval extending below u either ends strictly below u or contains u. By the rank defining u, fewer than $N/4$ intervals end strictly below it. Thus at most $(1/4 + \tau)N$ intervals extend below u. Similarly, at most $N/4$ intervals begin strictly above v, so at most $(1/4 + \tau)N$ extend above v.

At least $(1/2 - 2\tau)N \geq N/4$ intervals therefore lie wholly inside $[u, v]$, since $\eta < 1/32$. Enclosure gives this interval mass at least $1/4$. Continuity forces $u < v$, so both success tests pass. □

D Selection guarantees and state updates

D.1 Rounding an anchor

For a nondegenerate closed interval $J = [u, v] \subset (0, 1)$, choose the largest power-of-two spacing h no greater than its length. The routine `RoundUp` rounds u up to a multiple of h:

$$h = 2^{-r}, \qquad h \leq |J| < 2h, \qquad c = h\lceil u/h \rceil. \tag{34}$$

Lemma D.1 (Rounding to a binary fraction)**.** *The value c returned by* `RoundUp`(J) *belongs to J. Writing $c = (2i+1)/2^{m+1}$ in reduced form, it satisfies*

$$|J| < 2^{-m}. \tag{35}$$

Proof. The rounded value obeys $u \leq c < u + h \leq v$, so $c \in J$. The choice of spacing gives $h \leq |J| < 2h$. Reduction of the fraction cannot increase its denominator, so $m + 1 \leq r$ and $|J| < 2h \leq 2^{-m}$. □

Lemma D.2 (Density in a central interval)**.** *Let X be uniform on a positive-volume node region. Suppose $J = [u, v] \subset (\ell_x, r_x)$ satisfies*

$$\mathbb{P}(X_x \leq u) \geq \tfrac{1}{4}, \qquad \mathbb{P}(X_x \in J) \geq \tfrac{1}{4}, \qquad \mathbb{P}(X_x \geq v) \geq \tfrac{1}{4}.$$

Then, for every $c \in J$,

$$f_x(c) \geq \frac{1}{16|J|}.$$

Proof of Lemma 3.3. The interval comparisons in Section 5 certify lower probability bounds. If the larger bound is at least $1/2 - \eta$, `Select` returns its answer, breaking a tie in favor of the first queried label. On failure it rounds in the middle half of a queried free coordinate's interval. For `ChooseAnchor`, a successful central test supplies the interval to round. On failure it too rounds in the middle half.

For a successful central test, Lemma D.2 gives $f_x(c) \geq 1/(16|J|)$. By Lemma D.1 and the refinement identity, $g = \log f_x(c) > m - 4$. For either type of failed test, Lemma C.1 gives $W_x < w_* = 2^{-2kB}$ for the selected coordinate. Write $L = \log(1/W_x) > 2kB$. The density estimate (22) gives $g \geq L - 2B > k$ throughout the middle half. That interval has length $W_x/2$, so $m < L + 1$ by Lemma D.1. Thus

$$g \geq L - 2B > (1 - 1/k)L > (1 - 1/k)m - 4.$$

Both intervals used for rounding lie strictly inside the coordinate's range. The enumeration bound comes from Theorem 5.1. Appendix E bounds the precision of the rounded endpoints by $\Lambda + \log E_k$. □

D.2 Record updates

Algorithm 6: `Normalize`(v): propagate interval bounds through the in-node DAG.

Input: A node v whose arcs or interval bounds have changed.
foreach *vertex x in topological order* **do**
  $\ell_x \leftarrow \max(\{\ell_x\} \cup \{\ell_y : y \to x\})$
foreach *vertex x in reverse topological order* **do**
  $r_x \leftarrow \min(\{r_x\} \cup \{r_y : x \to y\})$

D.3 Consistency and termination

Proof of Proposition 3.5. Initially every node has positive volume, every order is represented, and the strict node graph bounds hold. We induct over completed calls and, within each call, over its changes to the state.

Before answering, a refinement deletes one vertex and adds an isolated vertex to a child, so the strict DAG bounds continue to hold. The anchor routine returns an interior position, as proved in Lemma 3.3. Lemma 3.2 preserves positive node volumes and consistency with the transcript. Normalization preserves the represented region by Lemma A.1.

If the queried labels lie in different child subtrees, their anchors determine the answer in every represented order. Otherwise, the answer guarantee in Lemma 3.3 ensure that the chosen answer retains at least $1/2 - \eta > 0$ of its node's volume. Recording its arc or interval bound therefore preserves positive volume and makes the answer true in every represented order. A new arc cannot create a cycle, since that would make the region empty. Normalization preserves this updated region. The transcript is then extended by the chosen answer.

During the subsequent repair, the temporary DAG bounds follow from Lemma 3.4. Each move again preserves positive volume and the transcript by Lemma 3.2. This applies to every vertex of a selected path, including those whose incident arcs were removed earlier in the batch. The full batch is completed, and Lemma 3.4 restores the strict bounds for the next call.

The finite certificate calculations and the explicit rounding rule terminate. As shown in Section 3.3, before answering each queried label can be refined only once: a second move would have gain zero, contradicting the failed-certification gain guarantee. Thus there are at most two such moves. A repair makes at most n further moves, so every call terminates with at most $n + 2$ refinements. All analytic guarantees used here apply to arbitrary positive-volume regions with the stated DAG bounds, independently of the interaction history. □

E Exact arithmetic and the stored state

The budget $E_k \le n$ bounds the number of digit assignments and their length. We verify that it also bounds their bit cost, and that the recursive state remains of polynomial size up to the comparison horizon.

E.1 Precision in the interval calculation

Suppose the stored endpoints have at most Λ fractional binary digits. An update takes maxima and minima, forms an integer linear combination, and divides by the power of two S. It therefore adds at most $\log S$ fractional digits. If there are b' interior vertices, each endpoint calculation makes $b't$ updates. The two calculations share digits but use their own values, so the precision in either is at most

$$\Lambda + b't \log S \le \Lambda + \log E. \tag{36}$$

This proves the precision claim of Theorem 5.1. All computed positions remain in $[0, 1]$, so their integer numerators need at most one additional bit. Digits and counts have at most $\log S$ and $\log E + 1$ bits, respectively.

E.2 Precision of new anchors

If a nondegenerate interval has dyadic endpoints with at most F fractional digits, its length is at least 2^{-F}. The rounding rule of Lemma D.1 therefore uses a grid exponent at most F, and its anchor has at most F fractional digits.

For a certified central interval, (36) gives $F \le \Lambda + \log E_k$. Forming the middle half of a stored interval gives endpoints with at most $\Lambda + 2$ digits. Since $E_k \ge 4$, both choices produce anchors with at most $\Lambda + \log E_k$ digits. This also covers the direct middle-half choice after a failed comparison certificate. The update of neighboring bounds and `Normalize` take maxima and minima of existing endpoints and the chosen anchor, adding no precision.

All initial endpoints are 0 or 1. After P refinements, induction therefore gives $\Lambda \le P \log E_k$, as claimed in (17). The bound includes the stored state during an unfinished call. Temporary endpoints in the next calculation add at most another $\log E_k$ digits.

E.3 Inventory and work per call

Each refinement creates at most one recursive-tree node. There are thus at most $P + 1$ nodes, n item records, and at most P stored child anchors. An item belongs to exactly one node graph, so their total numbers of vertices and arcs are at most n and n^2, respectively. The recorded prefix contains at most $Q(n)$ comparisons. Labels, pointers, depths, and counters have polynomial length, as do all dyadic endpoints by (17).

For one certificate, the algorithm enumerates at most $E_k \le n$ digit assignments. Each assignment uses two endpoint calculations, each with at most $b_k t_k \le \log E_k \le \log n$ updates. The boundary vertices remain fixed. Maxima, minima, integer multiplication, and division by powers of two operate on polynomially many bits. Central certification sorts at most n lower and upper endpoints and counts the intervals inside its candidate. Rounding computes a grid exponent and a ceiling using the same bounded-precision integers. Standard binary arithmetic therefore gives a fixed polynomial bound for each operation.

Before selecting an answer, a call makes at most two failure refinements and three calls to `Select`. Failure refinements use the middle half directly. After recording the answer, the single structural repair makes at most n refinements and hence at most n calls to `ChooseAnchor`.

The remaining work is polynomial in the stored graph size. Topological passes normalize intervals and compute directed-path lengths. A finite graph search finds each retained neighborhood; it stops when either the prescribed radius or all available vertices have been reached. Routing uses the recursive tree, and deterministic choices can be made by scanning labels. At the horizon, topologically sorting the bounded transcript DAG produces the fixed ranks for all later answers. Thus the bit-operation bound has a fixed exponent, independent of k and of the number of queries already answered.

F Parameter reference

Table 1 collects the parameters of the adversary and its analysis. The last two columns distinguish where a parameter's role is introduced from where its value is chosen and explained. All choices are fixed before the first query. Appendix B gives explicit choices for the digit alphabet, number of sweeps, and neighborhood radius in the local enclosure theorem.

Table 1: Parameters and their roles. References lead to the explanations in the text.

Symbol	Value	Role	Introduced	Choice
k	Integer, $k \geq 64$	Repair a vertex that reaches this degree after an answer.	§3.3	§4
H	2^k	Number of vertices refined in a selected directed path.	§3.3	§3.3
B	$(k+1)^{2H+1}$	Bounds the number of other vertices comparable with one vertex in its node graph.	App. C	App. C
w_*	2^{-2kB}	Calibrates the finite calculations: a coordinate of at least this width guarantees certification.	App. C	App. C
η	$1/(100k)$	Accept an answer whose certified probability is at least $1/2 - \eta$.	§3.3	§3.3
$Q(n)$	$n \lceil \log n \rceil$	Number of answers before switching to a fixed consistent order.	§3.3	§3.3

G Proofs of the consequences

G.1 Producing a target order

Proof of Corollary 7.1. A successful output assigns each key to one slot of R. Every total order satisfying the transcript must respect this assignment, so $T_q \leq e(R)$. Apply Theorem 4.1 with the parameter $k = k(n)$ selected in Section 6.2. It gives $q \geq I(R)/(1 + \delta_{k(n)})$. Take $\varepsilon_n = \delta_{k(n)}/(1+\delta_{k(n)})$ when a parameter qualifies and $\varepsilon_n = 1$ otherwise. Since $k(n) \to \infty$, $\varepsilon_n \to 0$, uniformly over targets. □

G.2 A compatible partition into layers

The reduction for an initial partial order needs an ordered partition whose layers retain many permutations. Cardinal et al.'s construction gives such a partition.

Lemma G.1 (Weak-order extension)**.** *Given a partial order P on n elements, one can deterministically construct in polynomial time a weak order W extending P such that*

$$\log e(W) \geq \log e(P) - C_0 n \log\log n$$

for an absolute constant C_0.

Proof. Cardinal et al. [14, Section 3, Lemmas 1 and 3] construct W in polynomial time with

$$I(W) \leq I(P) + 2n \log(I(P)/n + 1) + O(n).$$

Since $I(P) \leq \log(n!) \leq n \log n$, the extra term is $O(n \log\log n)$. Substitute $I(W) - I(P) = \log e(P) - \log e(W)$. For two elements, take $W = P$. □

An arbitrary partition into antichains need not admit a compatible layer order. For example, the relations $a < b$ and $c < d$ permit the antichains $\{a, d\}$ and $\{b, c\}$, but require opposite orders between them. Lemma G.1 supplies both the partition and a layer order respecting all initial relations.

Proof of Corollary 7.2. Construct W using Lemma G.1, and let its layers have sizes $m_1, \ldots, m_t$. Choose $k = k(n)$ once, using the full input size. Within every layer of size at least two, run a copy of the ordinary adversary with these parameters and its local horizon $Q(m_i) = m_i \lceil \log m_i \rceil$. A singleton needs no comparisons. Answer comparisons between layers from their fixed order.

Each local transcript has a consistent total order. Concatenating these orders gives a linear extension of W and hence of P, proving legality. At termination, each layer must have a unique possible order: two possible orders in one layer, combined with the same orders in the others, would give two global orders consistent with the full transcript.

Let q_i count comparisons within layer i. Theorem 4.1 applies to the local transcript with $T_{q_i} = 1$ and gives

$$q_i \geq \frac{\log(m_i!)}{1 + \delta_k}.$$

The inequality also holds for a singleton. Since $\sum_i m_i = n$ and $e(W) = \prod_i m_i!$, summing gives

$$\begin{aligned} q \geq \sum_i q_i &\geq \frac{\log e(W)}{1 + \delta_k} \\ &\geq \frac{\log e(P) - C_0 n \log\log n}{1 + \delta_k}. \end{aligned}$$

As $k(n) \to \infty$, this implies the stated bound, uniformly over P.

The common choice of $k(n)$ is essential to this estimate: each layer has the same δ_k, even when its size stays small. It also preserves polynomial time in n. Each finite calculation enumerates at most $E_k \leq n$ assignments. Across all layers, the horizons sum to at most $n \lceil \log n \rceil$, and the refinement and precision bounds of Section 6 apply with the ambient size n. The construction of W is polynomial-time as well. For the finitely many sizes at which no tuple qualifies, use a fixed topological order of P. □

References

[1] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching.* Second edition, Addison-Wesley, 1998. Publisher's page.

[2] Michael L. Fredman. *How good is the information theory bound in sorting?* Theoretical Computer Science 1(4), 355–361, 1976. doi:10.1016/0304-3975(76)90078-5.

[3] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[4] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[5] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[6] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[7] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[8] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[9] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[10] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[11] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[12] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.

[13] András Prékopa. *On logarithmic concave measures and functions.* Acta Scientiarum Mathematicarum 34, 335–343, 1973. Publisher's paper.

[14] Jean Cardinal, Samuel Fiorini, Gwenaël Joret, Raphaël M. Jungers, and J. Ian Munro. *An efficient algorithm for partial order production.* SIAM Journal on Computing 39(7), 2927–2940, 2010. doi:10.1137/090759860. Authors' manuscript.

[15] Bernhard Haeupler, Richard Hladík, John Iacono, Václav Rozhoň, Robert E. Tarjan, and Jakub Tětek. *Fast and simple sorting using partial information.* Algorithmica 88, article 41, 2026. doi:10.1007/s00453-026-01387-3.

[16] Ivor van der Hoog and Daniel Rutschmann. *Tight bounds for sorting under partial information.* Proceedings of the 65th IEEE Annual Symposium on Foundations of Computer Science (FOCS), 2243–2252, 2024. doi:10.1109/FOCS61266.2024.00131.

[17] Daniel Rutschmann. *Sorting under partial information with optimal preprocessing time via unified bound heaps.* arXiv:2604.12653, 2026. Author's manuscript.

[18] Ivor van der Hoog, Irina Kostitsyna, Maarten Löffler, and Bettina Speckmann. *Preprocessing ambiguous imprecise points.* 35th International Symposium on Computational Geometry (SoCG), LIPIcs 129, 42:1–42:16, 2019. doi:10.4230/LIPIcs.SoCG.2019.42.

[19] Ivor van der Hoog, Irina Kostitsyna, Maarten Löffler, and Bettina Speckmann. *Sorting under partial (interval order) information.* Journal of Computational Geometry 15(1), 143–171, 2024. doi:10.20382/jocg.v15i1a6.